System Design Document

Unified Service Scheduler

1. Problem Statement

The goal is to design an appointment scheduling system for automotive dealerships that replaces manual booking workflows.
The core challenge is not simply storing appointments, but ensuring that each booking is assigned both:

- an available service bay
- a qualified technician
  for the full anticipated service duration, while preventing double booking under concurrent requests.

2. Functional Requirements

Customers can request a service appointment by selecting:

- dealership
- service type
- date

The system must return valid appointment slots in real time.

A slot is only valid if:

- a compatible service bay is available
- a qualified technician is available

The system must prevent double booking under concurrent requests.

On confirmation, the system persists a valid appointment record.

3. Non-Functional Requirements

Consistency: prevent double booking

Scalability: support multiple dealerships

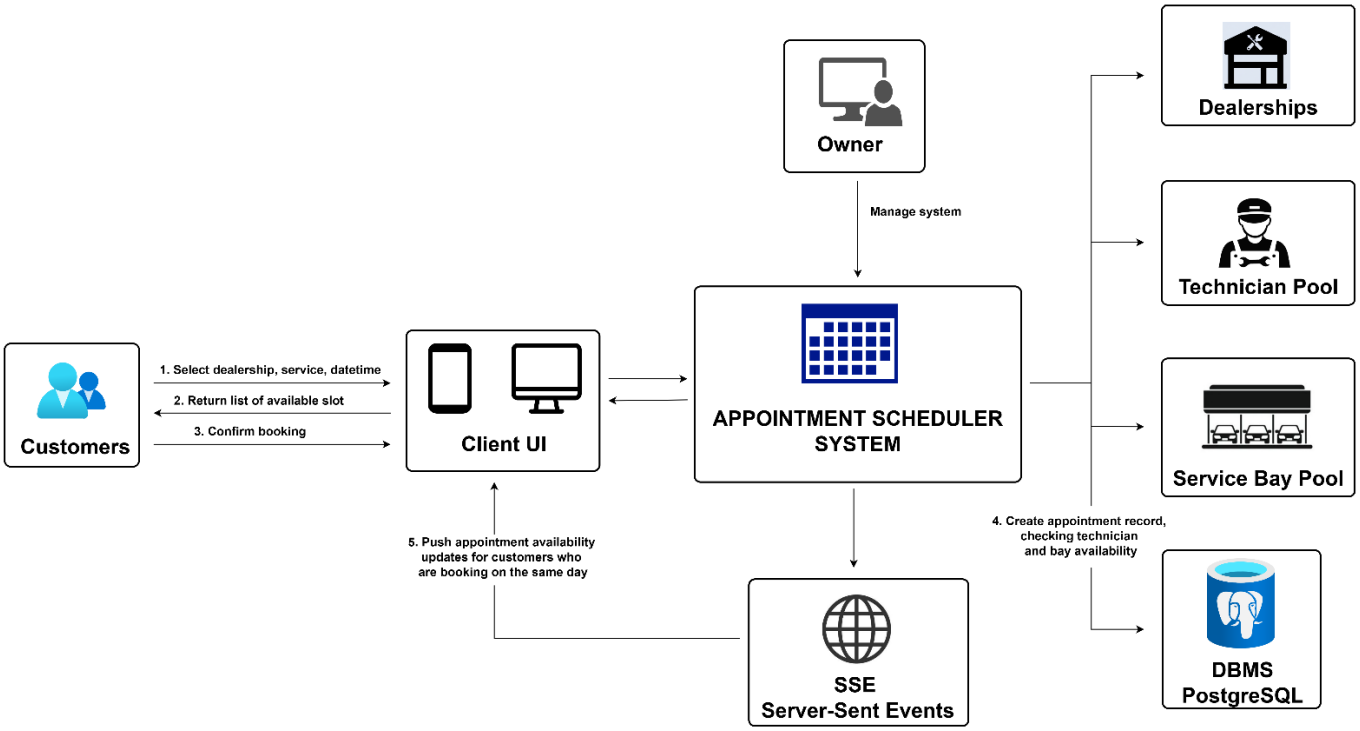
Low latency: slot lookup should be fast

Reliability: booking confirmation must be transactional

Observability: booking decisions must be traceable

Maintainability: modular scheduling components

4. Architecture Overview



5. Core Components

Availability API

Returns soft-availability appointment slots for a selected dealership, service, and date.

Booking API

Accepts booking confirmation requests and triggers transactional scheduling.

SSE (Sever-Sent Event)

Pushes real-time availability invalidation events to subscribed clients.

Availability Engine

Computes candidate appointment slots by checking technician and service bay availability.

Scheduling Engine

Matches a qualified technician and compatible service bay for a selected slot.

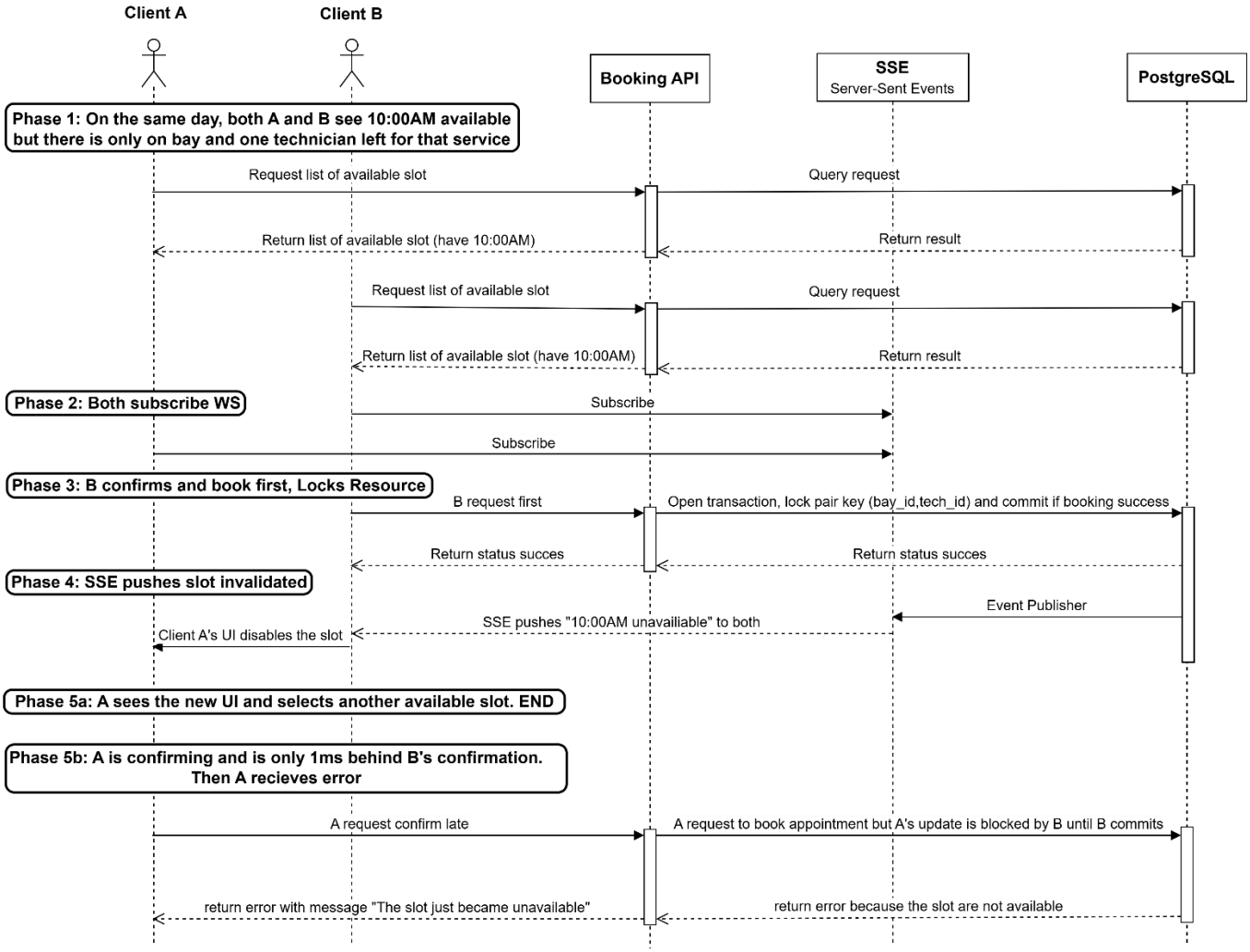
Booking Orchestrator

Performs final availability re-check, locking, appointment creation, and event publishing.

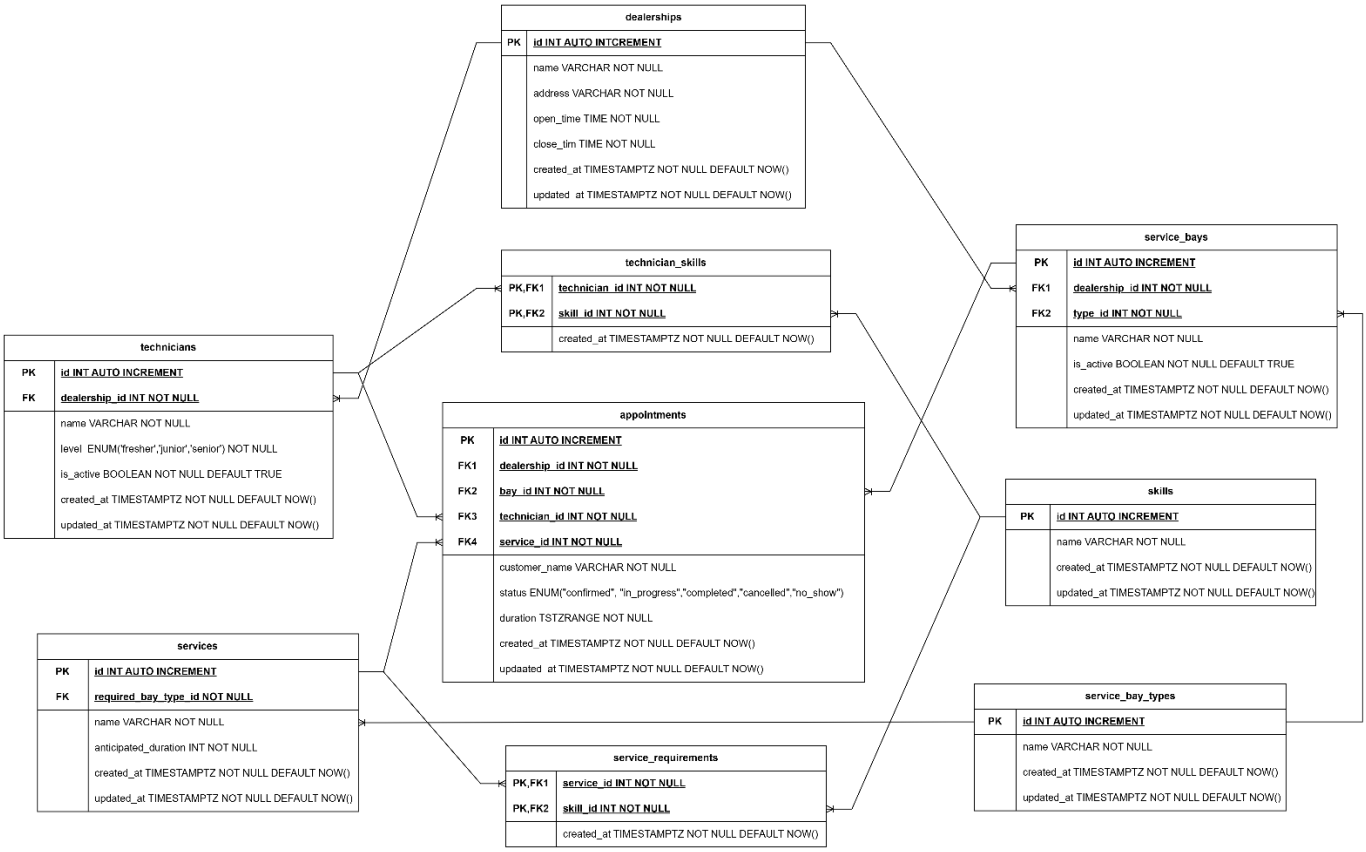
PostgreSQL

System of record and concurrency control boundary.

6. Data Flow & Concurrency Control Strategy



7. Data Model



appointments = booking ledger

technicians = human resources

service_bays = facility resources

skills / technician_skills = technician qualification

service_requirements = service skill requirements

service_bay_types = facility classification

dealerships = scheduling boundary

8. Technology Choices

Technology	Purpose	Why
Go	Backend API	Strong concurrency, simple deployment
PostgreSQL	Primary database	ACID transactions, row locking, advisory locks
SSE	Realtime updates	Push slot invalidation events and bandwith cost cheaper than Websocket
Docker	Local/dev environment	Reproducible setup
Postman	Test API	Familiar with me

9. GenAI Design Collaboration

Used GenAI to challenge architectural assumptions

Explored tradeoffs in:

- optimistic vs pessimistic booking
- bay classification modeling
- invalidation patterns by SSE

Independently validated all architectural decisions

Final design choices were selected based on domain fit, consistency guarantees, and implementation feasibility

Source Code: https://github.com/DannyTuanAnh/appointment-scheduler-application

Video Submission: https://www.youtube.com/watch?v=BAfZ4cZ5yDA